

ECE346 Final Report

Wilson Moyer, Natalia Kiernia, Ryan Zimmitti, George Kopf

Design overview [1 pt]: What components did you implement, and what techniques did you apply?

Our system used two ILQR functions running concurrently. The first ILQR ran as a safety monitor. From an input, the monitor considered the human input and applied the 5D bicycle kinematics to predict the vehicle's trajectory under human control. At the termination of a 12 step (1.2 s) dynamic trajectory simulated from the input control, the monitor would run receding horizon ILQR from the end of the trajectory. If the monitor predicted that the simulated dynamic trajectory or ILQR entered a failure state, the monitor would consider the human's input to be unsafe, and revert to the fallback policy.

The fallback policy consisted of following the control policy of the second ILQR continually running in the background. By itself, it can easily navigate around the track endlessly. It can also recover from perturbations as it drives. Whenever the first ILQR system detects that the human input would lead to an unsafe state (i.e. a state in which it is not possible to avoid entering a failure state in the future), it disconnects the human controller from the car's action, instead allowing the second ILQR system to take control of steering. This ILQR system attempts to steer the car to avoid entering an unsafe state (thus staying in a safe state) before relinquishing control back to the human driver.

ILQR requires a waypoint to base its reference path on. We needed a way to standardize the placement of waypoints to continually generate useful reference paths for the ILQR to base on. We ultimately decided to generate waypoints based on the car's current position in a way that the goalpost would be far from the car. This allowed the ILQR pathfinding to track through the center lane, which gave our car the ability to enter the center circle without freaking out. When the car came within a certain distance of the waypoint, we would regenerate the goalpost across the map.

Our safety filter identified failure states with three parts. The first was the lanelet boundaries generated by the provided ILQR code. The second was using hardcoded geometries representing the track as a square with rounded corners. The third was using SLAM's apriltag recognition to identify obstacles

and represent them as square failure regions. The car itself used a bounding box model to identify its own physical shape, allowing it to detect overlap in a more accurate way than a simple spherical representation.

As will be discussed below, on Demo Day we found the car did a strong job except for handling obstacle navigation. When presented with obstacles that were impossible to avoid, our car had no “stop” policy to fallback to, meaning we ended up driving through the obstacles or diving off road to avoid them as ILQR tried and failed to identify a safe path. After demoing with this setup, we went back and added in a second layer of safety filter to help rectify this imperfect obstacle-avoidance behavior.

This “safety filter on the safety filter” identifies when the car is in a state within the unsafe set of states (i.e. all states in which it is not possible to avoid entering a failure state in the future). In this scenario, ILQR’s best suggestion still drives us to a failure set as the safety filter takes control. We initiate breaking instead. In theory, we should never reach an unsafe set where failure is inevitable, because the safety filter should engage *before* the car ever gets that far. This second filter was a quick and dirty “bandaid” fix that allowed the car to perform significantly better in the obstacle section on Demo Day. When the car did find itself in an unsafe set, now it had the capability to come to a complete stop rather than be forced into a failure state. This will be further elaborated on in the outlook section.

ROS architecture [1 pt]: What are the main nodes carrying out the functionality and what topics and/or services do they use to communicate?

There are many nodes and topics we used in our architecture to implement our desired movement, control, and visualizations. The portion we worked the most on was of course `safety_filter_node`, where we create subscriptions to `teleop_topic` (type `AckermannDriveStamped`), `odom_topic` (type `Odometry`), `path_topic` (`PathMsg` type), and `static_obstacles_topic` (`MarkerArray` type). `teleop_topic` is used to get joystick input and gives us the human’s desired control for the truck. `odom_topic` sends type `Odometry` which gives us insight into the truck’s current state including position and velocity. When finding the

calculated path for a reference trajectory, we use `path_topic`. For obstacle detection, we use `static_obstacles_topic`'s `MarkerArrays` showing the position and size of static obstacles.

Taking all the information from the topics, we then publish to `drive_topic`, `goal_topic`, `traj_topic`, and `vis/FRS`. `drive_topic` is what the body of the calculations goes into, as this is where we set the speed and trajectory of the truck (type `AckermannDriveStamped`) which are fed into `drive_to_servo_node` where the data is actually used to move the truck. We also update `goal_topic` (`/goal_pose` with type `PoseStamped`) which sends updates to `routing_node` for replanning. `traj_topic` (`/Planning/Trajectory`) publishes the Path for visualization in ROS. For dynamic obstacles, we also publish to `vis/FRS` to see the forward reachable set of the dynamic truck in our extra-credit portion of this assignment.

Result summary [2 pt]: How did your solution perform at the task? Were there unexpected strengths or hurdles?

Generally our truck performed to expectations. In the initial tests with corner-turns, we observed successful navigation with minimal drift outside the lane boundaries. Our pessimistic bounding box and aggressive ILQR combined allowed us to do very well here, and as long as the localization holds, our truck demonstrates excellent lane assist. Unfortunately in our first runthrough with obstacles, the truck did not perform as well as expected. It was unclear to the team until other groups started testing that we needed better stopping behavior when encountering obstacles, as our ILQR would aggressively try to find a path around obstacles at the expense of ignoring lane boundaries. Additionally, because the truck is large, despite us navigating around obstacles we would occasionally bump into them. The bounding box was supposed to help with this behavior, but better tuning was needed before this feature worked well.

After some adjustments including a stopping behavior implementation, we performed much better on the obstacles. We programmed a preference to stop rather than drive through lane boundaries (essentially a safety filter for our safety filter), and successfully navigated around some of the obstacles.

Something we did not expect to behave as well as it did was the inner-lane turning behavior. As it was somewhat unclear that we were going to be testing the transition between outer and inner lane until

test day, some last-minute changes were implemented to allow the truck to navigate the inner track. While most groups struggled to navigate this turn, our truck was able to steer into the inner track successfully and maintain a safe distance from the inner walls. This can be attributed to our bounding box keeping the truck within the lane limits while our path planner allowed for inner track navigation.

Method assessment [3 pt]: Why did you choose this approach? Did your implementation deliver on the expected features? Is there anything you would do differently based on what you learned?

Our system implemented a predictive safety filter using two concurrently running ILQR controllers. The first controller considered the human input and extended it forward over a finite horizon using the vehicle dynamics model. The purpose of this controller was to predict where the human input would send the car, allowing for the identification of safe and unsafe controller inputs. This ILQR safety monitor looked ahead to determine when the safety filter needed to take control. Unsafe states were defined as states in which it was no longer possible to avoid entering a failure state in the future. This approach was motivated by reach-avoid safety filtering methods, which seek to minimally intervene while still ensuring that the system avoids unsafe regions and returns to a safe operating condition.

The second ILQR controller continually ran in the background and served as the core of the safety filter. It could navigate the track autonomously and recover from disturbances while driving the car. Usually, the human controller operated the car freely, but when the first ILQR detected that the input of the human controller would result in an unsafe state for the car, it disconnected the human controller from the car action and switched to the second ILQR controller to execute the action. This second ILQR controller generated a safe recovery path to keep the car in a safe state until handing the control back to the driver. The second ILQR controller minimized the trajectory cost function based on minimizing the deviation from the lane center, punishing unsafe states, and promoting smoother steering actions. For the second ILQR controller, the dynamically generated waypoints were needed to generate stable trajectories. As the human controller was expected to be the dominant controller in the proposed control scheme, the definition of waypoints manually was not feasible. Therefore, the system generated the waypoints on the

outer lane side of the car's current location. This promoted trajectories following the middle lane and ensured a smooth transition between the outer and inner lanes of the track.

Failure-state detection combined three independent safety representations. First, lanelet boundaries generated by the provided ILQR framework were used to define road constraints. Second, additional hardcoded geometric boundaries modeled the track as a square with rounded corners. Third, static obstacles detected through AprilTag-based SLAM localization were represented as obstacle exclusion regions. The vehicle itself was modeled using a rectangular bounding box rather than a point representation, allowing more accurate collision checking with lane boundaries and obstacles.

In the process of obstacle testing, there were instances in which it wasn't possible to create a safe trajectory around the obstacle. In such instances, the backup ILQR would sometimes opt to steer clear from the obstacle by heading towards another failure area, for example off-road. As a solution to this problem, a braking safeguard was introduced in the safety filter framework, which would check whether the backup ILQR's own planned trajectory would end up in the failure area. When this was determined, the truck would apply the brakes instead of following its ILQR trajectory. Although the solution came later in development, it proved to be quite effective in improving obstacle handling capabilities of the truck.

For the extra credit component, we incorporated into our system a simulated dynamic obstacle represented as a "phantom" vehicle within the traffic simulation environment. The phantom vehicle was treated as a moving obstacle whose simulated forward-reachable set was incorporated into the safety filter's failure-state evaluation. Although the obstacle existed only in simulation, the truck responded as if it were a real dynamic vehicle by modifying its trajectory to maintain safe separation distances.

Overall, the implementation, we believe, did a good job of achieving the desired lane following and safety intervention behaviors while also ensuring the human operator could continue to control the vehicle most of the time. The predictive safety filter was especially useful for avoiding unsafe lane changes and recovering from disturbances. Obstacle avoidance showed that the fallback ILQR algorithm tended to prioritize movement over a guaranteed safety constraint in some cases. If we had more time to

develop this project, we would restructure the fallback controller to prioritize safety more strongly and incorporate stopping behavior directly into the trajectory optimization framework rather than relying on a secondary braking safeguard.

Team plan assessment [1 pt]: What team member(s) took on which responsibilities? Did your development and testing process and timeline proceed as planned? How did you adapt your plans?

Our team worked collaboratively on most of the responsibilities: our workflow usually started with group discussion, and then progressed to team coding, where one team member would “drive,” writing the code, while one or more other members would “navigate,” assisting with high-level structure and catching mistakes in the coding process. This team process ended up being very effective in producing quality and conceptually sound code for the project. The timeline proceeded approximately according to plan: our team was glad that we started work early (the day after we received our feedback on our proposal), because the coding took much longer than we expected. Different project members were responsible for ensuring the completion of individual parts of the project, but all work was done collectively. This process helped us adapt to challenges in the process that we didn’t anticipate (which mostly included minor issues with our code and more parameter tweaking than we expected), and ultimately finish our project on time.

As per the project proposal, Natalia’s responsibility was to make sure we did proper literature review and translate that to workable methods for choosing good design plans, as well as work on tuning methods. George’s responsibility was to establish the ROS architecture and make sure topics and subscribers were properly set up. Wilson’s responsibility was to oversee the integration of the ILQR code to work for the safety value function, and implement it into a new safety-oriented trajectory planning algorithm. Ryan’s responsibility was to make sure the safety filter logic properly predicted the car’s dynamics and work on tuning the safety filter override. Despite individual responsibilities, we made sure all group members worked on every part together, helping ensure the entire group was on the same page throughout the project.

Outlook [2pt]: What are the remaining challenges in getting your solution to work truly smoothly? How would you envision solving them (e.g., if you had a whole year to work on it, or perhaps longer)? Did you encounter any challenges that seemed fundamental and therefore impossible to solve?

Challenge 1: Overintervention

One design challenge that our vehicle struggled with was that it was overprotective, occasionally taking control when the user's input action would result in a safe future state. This was likely due to issues with the bounding box and the double safety filter that we implemented. In order to maintain the safety of the vehicle, we considered any state in which any segment of the car's rectangular bounding box entered a failure region (either off the track or intersecting with an obstacle) to be a failure state. This was important to protect the car in some regions, specifically to prevent it from colliding with an obstacle or driving off the road on the side where the road was directly adjacent to the wall, but ended up being restrictive in other regions where the bounding box of the car was close to the width of the road, including the one-lane region at the top of the map and the region in the center of the map. In these restrictive regions, the safety filter would frequently consider any user action to be unsafe given that it was very difficult to produce an action that wouldn't result in some small segment of the car being off of the road, and would therefore rarely permit the user to have control in these regions. A possible solution to this would be a better definition of the failure set, allowing for more leniency in areas with looser boundaries and maintaining the same strict control in regions with tighter boundaries.

In addition, the double safety filter, which would determine whether the ILQR fallback policy would enter a failure set and intervene by applying the brakes if it detected a failure, was relatively elementary, given that it was a tweak that we decided to apply on the demonstration day. Our implementation simply tested whether the ILQR would enter a failure state within the next 10 time steps (1s) and apply a braking command if so. A more robust version of this filter could instead test whether the truck would enter a failure state if it first followed a set number of ILQR time steps and applied the brakes at the end of that time period, and if not, it would apply the brakes in the current time step. This second

filter would ensure that the ILQR could still run when possible, but the car would never enter a state in which it could not safely come to a stop. An even more advanced version of this would be to reduce the double safety filter into a single safety filter with a safety guarantee at the end of the ILQR trajectory. This could be done by running ILQR and forcing the end of the trajectory to be in a stopped state (by increasingly penalizing any non-zero velocity as t approaches T). This would result in a way to ensure that the car is always able to enter a “permanently safe” state (i.e. stopped) at the end of each fallback trajectory, rather than having to rely on the double safety filter to maintain vehicle safety.

Challenge 2: Obstacles

Another major challenge that our safety filter encountered was obstacle avoidance. While the truck was generally very successful at avoiding obstacles, the ILQR occasionally chose to avoid an obstacle by entering a different failure state, usually by driving off of the road. This seemed to be primarily a result of the ILQR that we were using, which was safety-oriented, but also likely too task-oriented than was helpful. Our project utilized an ILQR that sought to continue driving in order to promote a desirable fallback policy of correcting the vehicle’s driving from entering an unsafe state rather than stopping the vehicle altogether. While this was particularly successful in avoiding attempts to drive off of the road, it did not work as well when avoiding obstacles, as the ILQR sought to continue driving rather than avoid a crash, and therefore would occasionally drive off the road to get around an obstacle or attempt to drive forward when there existed no safe path forward. One way we attempted to address this was to increase the ILQR time horizon to increasingly penalize the truck for going out of bounds and to permit it to locate a more safe trajectory, but this practice produced diminishing results, as with each increase of horizon, the latency increased, resulting in other unsafe behaviors. The main way we ultimately addressed this was with our double safety filter, but this produced issues as described above. A better method to address this would be to restructure the ILQR in order to prioritize safety firstly, and consider staying on task or progressing forward as a secondary objective (with much less associated cost weight). We initially planned to do this and structure our ILQR based on the safety value function, but

ultimately did not have sufficient time to do so: given more time, we could implement a safety value function ILQR or hybrid ILQR between being both safety and progression oriented.

Fundamental Challenges:

One challenge that remained in our project was the SLAM localization and software to hardware control flow. At times, the vehicle lost localization, resulting in the ILQR struggling significantly to produce a policy that would safely guide the truck. Additionally, at times the car would struggle with transferring the control of the computer to a response, occasionally accelerating backward when a forward throttle control had been applied. These issues were mostly out of the scope of this project, but it's possible that given enough time, we could have possibly adjusted the truck to use its dynamics to determine when it was incorrectly localized and adjust accordingly.

Conclusion:

Overall, we enjoyed working on this project, and were happy with the progress that we were able to make on producing a safety filter. While the truck had some residual issues that we could still work on, it was fun to make something that could promote user safety in a large array of scenarios.